

The Agent-Readiness of LLM-Synthesized API Tool Implementations: A Comprehensive Survey of Benchmarks, Synthesis, and Evaluation Methodologies

The paradigm shift from Large Language Models (LLMs) as passive text generators to active cognitive controllers necessitates a standardized interface for interacting with the external digital environment. This interface, increasingly formalized through protocols such as the Model Context Protocol (MCP), relies on tool implementations that act as the sensory and motor organs of the agent.¹ As empirical research moves toward evaluating the "agent-readiness" of these tools, it becomes clear that task success is not merely a function of the agent's reasoning capability but is deeply contingent upon the quality of the tool implementations it consumes.³ The following report surveys the current literature to identify how tool quality, synthesized from documentation, determines task success and characterizes the specific properties that drive performance variance.

The Landscape of LLM Tool Use Benchmarking

The evaluation of tool use has evolved from verifying the syntactic correctness of a single function call to assessing the long-horizon orchestration of multiple, interdependent APIs within stateful environments.⁵ This evolution reflects a growing understanding that the "tool implementation" itself is a variable that can either facilitate or bottleneck agent performance.⁴

Comparative Analysis of Established Benchmarks

Existing benchmarks differ primarily in their scope, the fidelity of their environment, and the degree to which they isolate tool quality from agent reasoning.

Benchmark	Scope and Task Complexity	Evaluation Focus	Primary Metric	Environment Fidelity
ToolBench	16,464 APIs across 49 categories	Planning, retrieval, and multi-step	Pass Rate & Win Rate (Preference) ⁹	Real-world RESTful APIs (often

	(RapidAPI) ⁷	execution ⁸		unstable) ⁴
BFCL (v1-v4)	Single-turn to holistic agentic evaluation ⁵	Correct tool selection and parameter filling ¹⁰	AST Substring Matching Accuracy ¹⁰	Static/Execution (mix of simulated and live) ⁵
Agent-Diff	224 tasks across 9 enterprise services ¹¹	Stateful productivity software workflows ¹²	State-Diff (Success via environment change) ¹³	Containerized replicas of real services ¹¹
RestBench	High-quality movie (TMDB) and music (Spotify) tasks ¹⁴	Interactive planning and response parsing ¹⁵	Task Completion Rate ¹⁶	Real-world RESTful APIs ¹⁴
NESTful	300 samples of nested API sequences ¹⁷	Interdependent parameter passing between calls ¹⁷	Completion of nested sequences ¹⁷	Executable and non-executable samples ¹⁷
ACEBench	Normal, Special (imperfect), and Agent categories ¹⁸	Atomic-level and robust tool usage ¹⁸	Problem Identification Accuracy ¹⁸	Sandbox environment ¹⁸

ToolBench represents the canonical reference for large-scale tool evaluation, employing a Depth-First Search-based Decision Tree (DFSDT) to annotate multi-step solution paths.⁵ However, its reliance on live RapidAPI endpoints introduces instability due to server-side errors and API drift, which led to the development of StableToolBench, utilizing virtual API servers to ensure reproducibility.⁴ This shift highlights a critical gap: when tools are synthesized from documentation, their "runtime health" is often ignored, yet it significantly impacts the agent's perceived capability.⁴

In contrast, the Berkeley Function Calling Leaderboard (BFCL) focuses on the "contract" between the model and the function.¹⁰ By using Abstract Syntax Tree (AST) matching, BFCL provides a scalable way to evaluate tool-calling precision without the overhead of execution, although it has recently expanded into "holistic agentic evaluation" to capture the reasoning required for stateful settings.⁵ The distinction between these benchmarks is crucial for the

proposed study: ToolBench measures the holistic system (retrieval + planning + execution), whereas BFCL isolates the model's ability to adhere to the tool's schema.⁷

Multi-step Orchestration and Nested Dependencies

As tasks move from single-step CRUD operations to long-horizon workflows, the "response structure" of the tool becomes a primary property of interest. NESTful specifically evaluates "nested sequences," where the output of one API call is the input to the next.¹⁷ This benchmark reveals that existing evaluations often treat isolated calls as the ceiling of capability, failing to capture the cumulative error propagation that occurs when tool A 's response is too verbose or poorly structured for an agent to extract the necessary parameter for tool B .¹⁷

Tool and API Synthesis from Technical Documentation

The process of automatically generating tool implementations—such as MCP servers—from documentation is a nascent field but builds on a history of Schema-Guided Dialogue (SGD) research. The goal of synthesis is to bridge the "ontology bottleneck," allowing models to interact with new services zero-shot by understanding natural language schema descriptions.²⁰

The Evolution from SGD to MCP

The principles established by Google's SGD framework in 2019 provided the theoretical foundation for contemporary protocols like the Model Context Protocol.²⁰ SGD demonstrated that a single model could generalize to new APIs by reasoning about machine-readable schemas.²¹ MCP operationalizes this by providing a universal standard for tool integration, shifting the burden from custom integrations to a standardized, two-way connection between hosts and servers.¹

Literature on synthesis, such as the MCPmed project, illustrates the practical implementation of these concepts in specialized domains like bioinformatics.²³ MCPmed provides templates (e.g., Cookiecutter) to accelerate the creation of MCP servers that associate web service endpoints with scientific metadata.²³ Evaluation in these contexts often moves beyond static analysis to "machine-actionability," where the synthesized tool is tested for its ability to enable autonomous, LLM-driven research.²³ A key innovation identified in this area is the use of "breadcrumbs"—lightweight HTML metadata systems that guide agents toward LLM-native services even when a full MCP server is not yet implemented.²³

Correctness Evaluation of Synthesized Tools

A central challenge in the user's proposed study is evaluating the "correctness" of an LLM-synthesized implementation. The literature suggests a hierarchy of evaluation methods:

- **Deterministic Schema Validation:** Checking if the synthesized tool conforms to standard JSON or OAS (OpenAPI Specification) schemas. This catches structural

corruption but misses semantic failures.²⁴

- **Execution-Response Matching:** Running the synthesized tool and comparing its output against a reference implementation.¹⁰
- **Agentic Tool Annotation:** A novel approach where a second agent (the "annotator") interacts with the synthesized tool to assess its health and collect response examples, effectively "verifying" the tool before it is used in a benchmark.⁴
- **Static Analysis vs. Oracle Execution:** Frameworks like SkillsBench and ProjectGen use "Oracle execution," where a reference solution must achieve a 100% pass rate in a containerized environment to validate the task's solvability.²⁶

The state of the art in synthesis, exemplified by ToolACE, utilizes "self-evolution synthesis" to curate a pool of over 26,000 diverse APIs, focusing on the diversity and complexity of the generated tool-learning data.⁶

The Fidelity Gap: Synthesized vs. Human-Authored Artifacts

A core component of the user's study design is the comparison between synthesized implementations and human-authored reference baselines. Empirical work consistently reveals a "quality gap" that manifests in downstream agent performance.

Curation vs. Self-Generation in Procedural Knowledge

The SkillsBench study provides the most direct comparison between human-authored and LLM-generated "skills"—structured packages of instructions and code used to augment agents.²⁶ The study found a substantial delta:

Condition	Success Rate Impact	Mechanism of Influence
Human-Authored Skills	+16.2 percentage points (average) ²⁷	Human editorial judgment omits redundant info and focuses on "how-to" guidance. ²⁹
LLM Self-Generated Skills	Negligible or negative (-1.3 points) ²⁹	Models fill knowledge gaps with "plausible-sounding but imprecise" procedures. ²⁷
No Skills (Baseline)	0.0 points	Agents struggle with domain-specific workflow

		constraints. ³⁰
--	--	----------------------------

This gap suggests that synthesized tools are prone to "over-comprehensiveness," where the inclusion of redundant documentation increases token usage by 20% and confuses the agent's reasoning process.²⁹ Furthermore, the study identifies that agents are "execution beasts" when given precise scaffolding but fail the "self-authoring loop," indicating that synthesized tools may lack the "portability" and "instructional clarity" that human authors naturally provide.³⁰

Patterns of Optimization and Maintenance

In software engineering tasks, human-authored performance pull requests (PRs) are 18% more likely to include explicit validation (benchmarking/profiling) than agent-authored ones.³² Agents tend to rely on static reasoning, whereas human experts provide the empirical evidence necessary for high-stakes tool use.³² This finding has direct implications for the user's study: a synthesized tool implementation may look "correct" on the surface but fail in a multi-step task because it lacks the robust internal validation or "error-handling transparency" that a human developer would implement.²⁴

Taxonomies of Agent-Ready Tool Properties

To operationalize "tool quality," several papers have attempted to classify the properties that make a tool "agent-usable." These properties transcend simple functional correctness and enter the realm of cognitive ergonomics.

Specification and Execution Dimensions

WildAgtEval proposes a two-dimensional taxonomy of real-world complexity that directly maps to tool properties ³³:

1. **API Specification Properties:**
 - **Schema Clarity:** Adherence to meticulous domain-specific formatting (e.g., "[shipping carrier]-[id]").³³
 - **Constraint Transparency:** Explicit documentation of usage rules that the agent must navigate.³⁴
 - **Documentation Density:** The "high content-to-noise ratio" of the page; overly long documentation leads to truncation where the agent misses the actual tool definition.³⁵
2. **API Execution Properties:**
 - **Response Structure (JSON Processing):** The degree to which responses are structured for easy extraction. Research shows that including a schema of the structured data in the prompt improves performance by up to 12%.³⁶
 - **Error Handling Transparency:** The ability of the tool to return meaningful errors that separate "schema violations" (structural) from "stale data" (semantic).²⁴
 - **Noisy Output Resilience:** The tool's propensity to return irrelevant or "sponsored"

results, which can reduce strong LLM performance by 27.3%.³³

Operationalizing "Agent-Ready" Quality

A tool is "agent-ready" if it minimizes the "semantic gap" between human intent and machine execution.²⁸ The literature identifies several "Tiers" of tool-result validation that define quality²⁴:

- **Tier 1: Deterministic Schema Validation:** Validating types and required fields before the response reaches the LLM.²⁴
- **Tier 2: Domain-Specific Assertions:** Encoding business logic invariants (e.g., verifying that a returned timestamp is within a requested window).²⁴
- **Tier 3: LLM-Based Semantic Verification:** Using a smaller model to audit if the tool response actually answers the query.²⁴

These tiers provide a rubric for Phase 4 of the user's study, allowing failing runs to be attributed to the absence of these quality gates.

Agent Evaluation Methodology for Multi-Step Tasks

Evaluating agents on multi-step API tasks requires moving beyond input-output matching to "trajectory evaluation," where the agent's reasoning, tool selection, and state changes are all scrutinized.³

Ground Truth and Verification Strategies

Best practices for ground truth in agentic systems involve three distinct types of graders³⁹:

1. **Code-based (State-Diff):** The "gold standard" for objective tasks. It measures whether the expected change in the environment state (e.g., a new Slack message or a database update) was achieved, rather than checking the agent's text response.¹¹
2. **Model-based (LLM-as-a-Judge):** Used for subjective qualities like "Plan Quality" or "Coherence".³⁸
3. **Human-in-the-Loop:** Essential for high-stakes tasks to calibrate the other graders.³⁸

Evaluation Aspect	Best Practice Methodology	Confound to Address
Tool Selection	Compare selected tool ID against a ground-truth mapping for the sub-task. ⁴¹	Conflating model planning error with ambiguous tool descriptions. ³⁹
Execution Efficiency	Compare the number of steps taken against the	Agents getting stuck in redundant API loops due to

	"Golden Path" (shortest path). ³⁸	poor tool feedback. ³⁸
Response Parsing	Evaluate whether the agent correctly extracted parameters from long JSON outputs. ³⁶	"Lost in the middle" phenomenon in long context windows. ⁴²

Confounds in Evaluation

A critical confound identified in the literature is "tool-reasoning entanglement".³ If a task asks for an action but doesn't specify a required parameter (e.g., a filepath), and the evaluation script expects a specific default, the agent will fail even if it is highly capable.³⁹ Mitigation strategies involve creating "reference solutions" (known working outputs) to prove the task is solvable and the grader is correctly configured.³⁹

Controlled Empirical Studies on Tool Conditions

The user's study design—varying tool quality as an independent variable—is supported by several papers that have run similar controlled comparisons.

Access to Documentation and Implementations

The Agent-Diff framework provides a direct precedent for Phase 3 and 4 of the user's study.¹¹ They evaluated 9 LLMs across 224 tasks using ablation experiments to assess the contribution of API documentation.¹¹ By comparing a "No-docs baseline" (where agents must discover endpoints) against a "With-docs" condition, they quantified how much implementation detail facilitates success.¹³

Mock vs. Real Environmental Factors

ClawsBench treats "scaffolding components" (like domain skill specifications) as experimental factors that are varied independently.¹² They found that "Progressive Disclosure"—providing API knowledge only as needed rather than overwhelming the context—was a critical factor in performance.¹² This aligns with the "response structure" and "schema quality" properties the user intends to characterize.

Study	Independent Variable	Key Finding
SkillsBench	No Skills vs. Curated vs. Self-Gen ²⁷	Human-authored guidance is a non-optional investment for reliability. ²⁹

SOP-Bench	Tool Registry Size ⁴³	When the tool registry is much larger than necessary, agents invoke incorrect tools nearly 100% of the time. ⁴³
RestBench/RestGPT	Static vs. Online Planning ¹⁴	Online planning based on tool feedback is required for real-world messy interactions. ¹⁵

Implications for the User's Empirical Study

The proposed four-phase study design is well-aligned with the frontier of agentic research, but the literature suggests several refinements to ensure validity.

Insights for Phase 1: Tool Synthesis

The production of MCP server implementations should leverage the "Seed Tool" principle.⁴ Since LLMs often synthesize "hallucinated" or non-functional APIs, the study could benefit from an automated "health annotator" phase to ensure that the baseline human tools and the synthesized tools are at least functionally operational before evaluation.⁴ The use of standard templates like the MCPmed "Cookiecutter" can standardize the synthesis process, reducing variance caused by the synthesis method itself.²³

Insights for Phase 2: Human-Authored Baseline

The human baseline should be characterized by "Editorial Judgment".²⁹ The literature shows that human authors tend to create "focused, modular" guidance, whereas LLMs generate "verbose, redundant" documentation.²⁶ To ensure the baseline is a true "upper-bound," authors should be instructed to follow best practices for "agent-friendly docs," such as using Markdown and providing clear "how-to" procedures rather than just raw DDL/OAS specs.³⁵

Insights for Phase 3: Agent Evaluation

Using a fixed ReAct agent (Claude/GPT-4o) is a sound strategy to isolate the impact of the tool.⁴³ However, the choice of "Agent Harness" (the code wrapping the agent) can be a hidden variable. Studies like ClawsBench show that the harness's ability to handle context pruning and safety gating can overshadow the model's raw capability.¹² The user should standardize the "tracing" of these runs to capture intermediate failures in the action-observation loop.³⁸

Insights for Phase 4: Property Characterization

The five properties identified (schema quality, error handling, response structure, coverage, path correctness) are supported by the literature, but they could be further operationalized:

- **Schema Quality:** Can be measured by the "Atomic-Level Accuracy" (how well it handles specific parameter types like enums or objects).¹⁸
- **Error Handling:** Should be characterized by "Error Source" (internal model-driven vs. external environment) and the agent's "Self-Critique" behavior when an error occurs.⁴⁸
- **Response Structure:** Can be quantified by "Token Efficiency" or the agent's "Intent Resolution" success when parsing complex JSON.³⁶

Research Gaps and Potential Contributions

While the survey identifies significant work in each dimension, the proposed study is uniquely positioned to address several unresolved questions:

1. **The Direct Correlation of Synthesis Artifacts to Success:** While SkillsBench compares "Skills," there is little work specifically comparing the *implementation* code of a tool (e.g., the Python logic of an MCP server) synthesized from documentation versus human-written code.⁶
2. **Tool Quality as a Continuous Variable:** Most studies treat tools as "present" or "absent." The user's goal to attribute failure to *specific properties* (like the verbosity of the response structure) would provide much-needed granularity to the field.²⁴
3. **MCP-Specific Agent-Readiness:** As MCP becomes the industry standard, empirical data on what makes an "MCP server" effective—beyond just adhering to the protocol—is highly valuable to the developer community.⁸

In conclusion, the success of LLM agents is as much an interface design problem as it is a reasoning problem. By systematically varying the tool implementation while keeping the agent fixed, the proposed study will provide a rigorous map of the "Semantic Gap," identifying the critical engineering patterns required to build a truly agent-native digital infrastructure. The integration of state-diff evaluation, fine-grained property taxonomies, and a focus on documentation-to-code synthesis positions this research at the forefront of the next evolution in autonomous AI.

Works cited

1. What is Model Context Protocol (MCP)? A guide | Google Cloud, accessed April 15, 2026, <https://cloud.google.com/discover/what-is-model-context-protocol>
2. Agentic Artificial Intelligence (AI): Architectures, Taxonomies, and Evaluation of Large Language Model Agents - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2601.12560v1>
3. The Necessity of a Unified Framework for LLM-Based Agent Evaluation - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2602.03238v1>
4. Learning to Rewrite Tool Descriptions for Reliable LLM-Agent Tool Use - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2602.20426>
5. A Comprehensive Survey of Benchmarks for Evaluating Tool and Function Calling in Large Language Models - Hugging Face, accessed April 15, 2026,

https://huggingface.co/datasets/tuandunghcmut/BFCL_v4_information/blob/main/A%20Comprehensive%20Survey%20of%20Benchmarks%20for%20Evaluating%20Tool%20and%20Function%20Calling%20in%20Large%20Language%20Models.md

6. Towards Reliable Benchmarking: A Contamination Free, Controllable Evaluation Framework for Multi-step LLM Function Calling - Semantic Scholar, accessed April 15, 2026, <https://www.semanticscholar.org/paper/Towards-Reliable-Benchmarking%3A-A-Contamination-for-Maekawa-Hassell/ffe85ca84d5f069ec54870cafd0f13cb6c4930ae>
7. ToolBench: LLM Tool-Use Benchmark - Emergent Mind, accessed April 15, 2026, <https://www.emergentmind.com/topics/toolbench>
8. ToolBench Evaluation: LLM Tool-Use Insights - Emergent Mind, accessed April 15, 2026, <https://www.emergentmind.com/topics/toolbench-evaluation>
9. ToolEval Leaderboard, accessed April 15, 2026, <https://openbmb.github.io/ToolBench/>
10. ICML Poster The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models, accessed April 15, 2026, <https://icml.cc/virtual/2025/poster/46593>
11. [2602.11224] Agent-Diff: Benchmarking LLM Agents on Enterprise API Tasks via Code Execution with State-Diff-Based Evaluation - arXiv, accessed April 15, 2026, <https://arxiv.org/abs/2602.11224>
12. ClawsBench: Evaluating Capability and Safety of LLM Productivity Agents in Simulated Workspaces - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2604.05172>
13. agent-diff-bench/agent-diff: Interactive Sandboxes for AI Agent Evaluations & Reinforcement Learning on 3rd party APIs (Slack, Linear, Box, gCalendar) - GitHub, accessed April 15, 2026, <https://github.com/agent-diff-bench/agent-diff/>
14. Yifan-Song793/RestGPT: An LLM-based autonomous agent controlling real-world applications via RESTful APIs - GitHub, accessed April 15, 2026, <https://github.com/Yifan-Song793/RestGPT>
15. RestGPT: Connecting LLMs with RESTful APIs, accessed April 15, 2026, <https://restgpt.github.io/>
16. Live API-Bench: 2500+ Live APIs for Testing Multi-Step Tool Calling - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2506.11266v2>
17. A Benchmark for Evaluating LLMs on Nested Sequences of API Calls - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2409.03797v1>
18. ACEBench: A Comprehensive Evaluation of LLM ... - ACL Anthology, accessed April 15, 2026, <https://aclanthology.org/2025.findings-emnlp.697.pdf>
19. Claude Mythos vs Claude Opus 4.6: How Big Is the Cybersecurity Capability Gap?, accessed April 15, 2026, <https://www.mindstudio.ai/blog/claude-mythos-vs-opus-4-6-cybersecurity-gap>
20. The Convergence of Schema-Guided Dialogue Systems and the Model Context Protocol: A New Paradigm for Agentic Interoperability - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2602.18764v2>

21. The Convergence of Schema-Guided Dialogue Systems and the Model Context Protocol - arXiv, accessed April 15, 2026, <https://arxiv.org/pdf/2602.18764>
22. Enhancing Model Context Protocol (MCP) with Context-Aware Server Collaboration - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2601.11595v2>
23. MCPmed: a call for Model Context Protocol-enabled bioinformatics ..., accessed April 15, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12927880/>
24. The Tool Result Validation Gap: Why AI Agents Blindly Trust Every API Response, accessed April 15, 2026, <https://tianpan.co/blog/2026-04-10-tool-result-validation-gap-agents-blindly-trust>
25. LLM Evaluation: Key Concepts & Best Practices - Nexla, accessed April 15, 2026, <https://nexla.com/ai-readiness/llm-evaluation/>
26. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2602.12670v1>
27. Benchmarking How Well Agent Skills Work Across Diverse Tasks - SkillsBench, accessed April 15, 2026, <https://www.skillsbench.ai/skillsbench.pdf>
28. Towards Realistic Project-Level Code Generation via Multi-Agent Collaboration and Semantic Architecture Modeling - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2511.03404v1>
29. Humans required: Why AI coding still requires human experts to work most effectively, accessed April 15, 2026, <https://adhoc.team/2026/03/09/humans-required-why-ai-coding-still-requires-human-experts-to-work-most-effectively/>
30. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks - alphaXiv, accessed April 15, 2026, <https://www.alphaxiv.org/overview/2602.12670v1>
31. Skillsbench Showed Models Aren't Good at Generating Their Own Skills : r/rajistics - Reddit, accessed April 15, 2026, https://www.reddit.com/r/rajistics/comments/1r77v1h/skillsbench_showed_models_arent_good_at/
32. How Do Agents Perform Code Optimization? An Empirical Study - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2512.21757v1>
33. A Comprehensive Evaluation of LLM Agents Under Real-World API Complexity - arXiv.org, accessed April 15, 2026, <https://arxiv.org/html/2601.00268v1>
34. Beyond Perfect APIs: A Comprehensive Evaluation of LLM Agents Under Real-World API Complexity - ResearchGate, accessed April 15, 2026, https://www.researchgate.net/publication/399438502_Beyond_Perfect_APis_A_Comprehensive_Evaluation_of_LLM_Agents_Under_Real-World_API_Complexity
35. Lms vs. agents as docs consumers - Dachary Carey, accessed April 15, 2026, <https://dacharycarey.com/2026/02/26/lms-vs-agents-as-docs-consumers/>
36. How Good Are LLMs at Processing Tool Outputs? - ACL Anthology, accessed April 15, 2026, <https://aclanthology.org/2026.eacl-long.134.pdf>
37. A Comprehensive Evaluation of LLM Agents Under Real-World API Complexity - arXiv, accessed April 15, 2026, <https://arxiv.org/html/2601.00268>
38. Agent Factory Recap: A Deep Dive into Agent Evaluation, Practical Tooling, and

Multi-Agent Systems | Google Cloud Blog, accessed April 15, 2026,
<https://cloud.google.com/blog/topics/developers-practitioners/agent-factory-recap-a-deep-dive-into-agent-evaluation-practical-tooling-and-multi-agent-systems>

39. Demystifying evals for AI agents - Anthropic, accessed April 15, 2026,
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
40. Production-Ready LLM Agents: A Comprehensive Framework for Offline Evaluation, accessed April 15, 2026,
<https://towardsdatascience.com/production-ready-llm-agents-a-comprehensive-framework-for-offline-evaluation/>
41. AI agent evaluation: A practical framework for testing multi-step agents - Articles - Braintrust, accessed April 15, 2026,
<https://www.braintrust.dev/articles/ai-agent-evaluation-framework>
42. Evaluating LLM-Generated Versus Human-Authored Responses in Role-Play Dialogues, accessed April 15, 2026, <https://arxiv.org/html/2509.17694v2>
43. SOP-Bench: Complex Industrial SOPs for Evaluating LLM Agents - Emergent Mind, accessed April 15, 2026,
<https://www.emergentmind.com/papers/2506.08119>
44. Benchmarking LLM Tool-Use in the Wild - OpenReview, accessed April 15, 2026,
<https://openreview.net/forum?id=yz7fL5vfpn>
45. ToolBench Dataset Overview - Emergent Mind, accessed April 15, 2026,
<https://www.emergentmind.com/topics/toolbench-dataset>
46. AI transformation is the new digital transformation. Your vendor stack isn't ready., accessed April 15, 2026,
<https://www.speakeasy.com/blog/agent-native-vendor-evaluation>
47. Generative AI Collaboration Platform - Orq.ai, accessed April 15, 2026,
<https://orq.ai/evaluation>
48. CRITICTOOL: Evaluating Self-Critique Capabilities of Large Language Models in Tool-Calling Error Scenarios - ACL Anthology, accessed April 15, 2026,
<https://aclanthology.org/2025.emnlp-main.1355.pdf>
49. Agent Evaluators for Generative AI - Microsoft Foundry, accessed April 15, 2026,
<https://learn.microsoft.com/en-us/azure/foundry/concepts/evaluation-evaluators/agent-evaluators>